Simulation

```
! pip install pandas numpy matplotlib seaborn
```

Show hidden output

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
import random, time
np.random.seed(6698)
random.seed(6698)
```

```
def generate_poisson_arrivals(n_customers, lam):
    """
    Generate cumulative arrival times for a Poisson process with rate lam.
    Interarrival times are i.i.d. Exp(lam); returns a nondecreasing array of length n_customers.
    """
    interarrival_times = np.random.exponential(scale=1/lam, size=n_customers)
    arrival_times = np.cumsum(interarrival_times)
    return arrival_times
```

```
def generate_uniform_interarrivals(n_customers, lam):
    """
    Generate uniform interarrival times on [0, 2/lam]
    Mean = 1/lam (same as exponential)
    """
    # Parameters for uniform distribution
    min_time = 0
    max_time = 2.0 / lam

    # Generate uniform interarrival times
    interarrival_times = np.random.uniform(min_time, max_time, size=n_customers)

    return interarrival_times

def generate_uniform_arrivals(n_customers, lam):
    """
    Generate cumulative arrival times with uniform interarrivals
    """
    interarrival_times = generate_uniform_interarrivals(n_customers, lam)
    arrival_times = np.cumsum(interarrival_times)
    return arrival_times
```

```
def generate_service_time(n_customers, mu):
    """
    Generate i.i.d. service durations S_i - Exp(mu) with mean 1/mu.
    """
    service_times = np.random.exponential(scale=1/mu, size=n_customers)
    return service_times
```

```
def simulate_single_server_fifo(arrival_times, service_times):
    """
    Simulate a single-server, FIFO, work-conserving queue.
    """
    n_customers = len(arrival_times)

    # initialize arrays
    service_start_times = np.zeros(n_customers)
    customer_departure_time = np.zeros(n_customers)
    customer_waiting_time = np.zeros(n_customers)
    system_times = np.zeros(n_customers)

    # processing the first customer process
    service_start_times[0] = arrival_times[0]
    customer_departure_time[0] = service_start_times[0] + service_times[0]
    customer_waiting_time[0] = 0  # No waiting time for the first customer
    system_times[0] = customer_departure_time[0] - arrival_times[0]

    for i in range(1, n_customers):
        service_start_times[i] = max(arrival_times[i], customer_departure_time[i-1]) # chooses the max between the two time
        customer_departure_time[i] = service_start_times[i] + service_times[i]
        customer_waiting_time[i] = service_start_times[i] - arrival_times[i]
        system_times[i] = customer_departure_time[i] - arrival_times[i]

    return service_start_times, customer_departure_time, customer_waiting_time, system_times
```

```
def theoritical_utilization(lam, mu):
    """
    Calculate theoritical utilization of a single-server queue for plotting the graph
    """
    rho = lam / mu
    return rho
```

```
def actual_utilization(arrival_times, service_times, customer_departure_time):
    """
    Calculate the actual utilization of a single-server queue for actual test
    """
    total_service_time = np.sum(service_times)
    simulation_start = arrival_times[0] #this should be zero
    simulation_end = customer_departure_time[-1]
    simulation_horizon = simulation_end - simulation_start
    utilization = total_service_time / simulation_horizon
    return utilization
```

```
def Lq_littles_law(lam, average_wait_time):
    """
    This is an approximation that works well for steady-state systems (theoritical)
    """
    Lq = lam * average_wait_time
    return Lq
```

```
def Lq_discrete_events(arrival_times, service_start_times, customer_departure_times):
    """
    This tracks the queue length changes throughout the simulation
    """
```

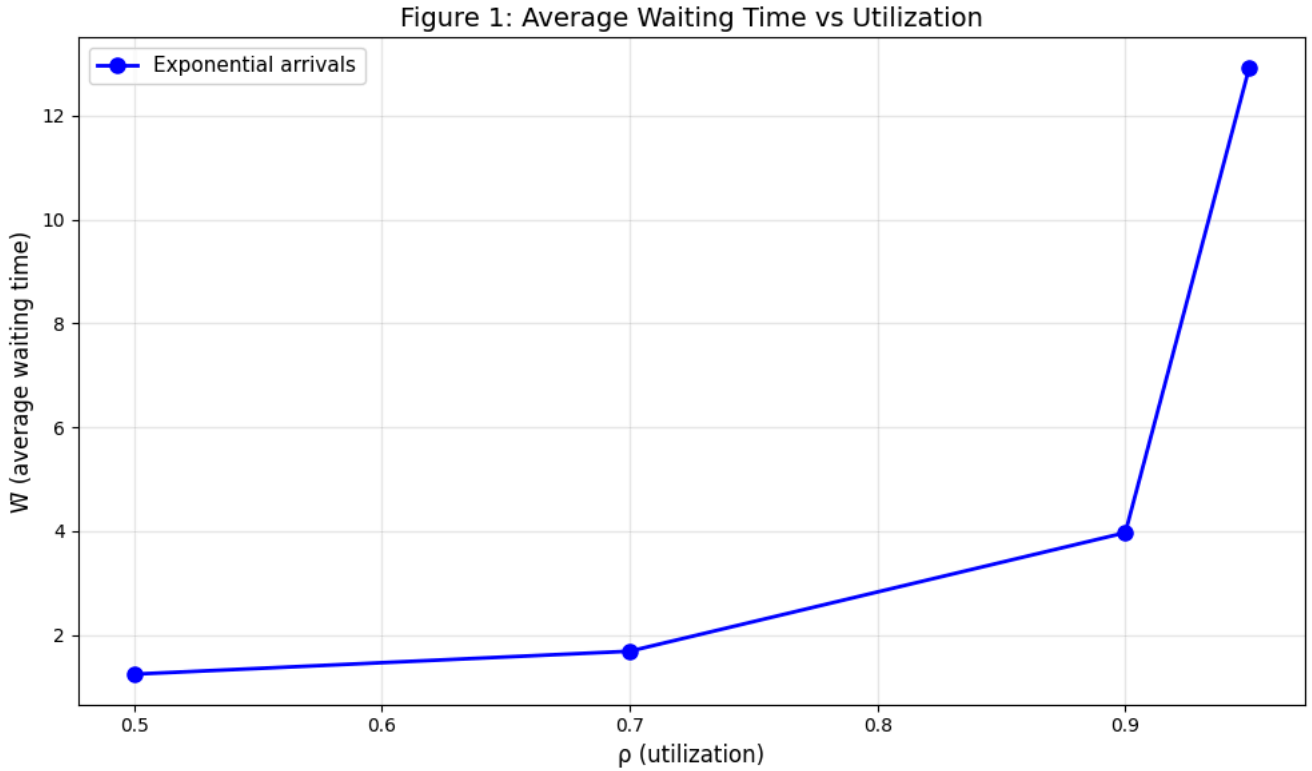
```
    # create a list of events in reference to their impact on the queue
    events = []

    for i in range(len(arrival_times)):
        # Customer arrives and adds to the queue length, +1
        events.append((arrival_times[i], 1))
        # Customer starts their service, queue length decreases -1
        events.append((service_start_times[i], -1))

    # Sort by time
    events.sort()

    # Initialize variables and calculate time-weighted average
    queue_length = 0
    total_customer_time = 0
    previous_time = 0

    for time, change in events:
        # update queue length
        queue_length += change
        queue_length = max(0, queue_length)

        # calculate time-weighted average, add area under curve for previous period
        total_customer_time += queue_length * (time - previous_time)

        previous_time = time

    # calculate average
    simulation_horizon = customer_departure_times[-1] - arrival_times[0]
    average_queue_length = total_customer_time / simulation_horizon

    return average_queue_length
```

```
def run_grid(pairs=((0.5,1.0), (0.7,1.0), (0.9,1.0), (0.95,1.0)), N=2000, arrival_type="exp"):
    util_theor_list = []
    util_pract_list = []
    wait_time_list = []
    sys_time_list = []
    Little_law_list = []  #
    average_queue_length_list = []
    table_data = []

    for lam, mu in pairs:
        # generate arrivals based on type
        if arrival_type == "exp":
            arrival_times = generate_poisson_arrivals(N, lam)
        elif arrival_type == "uniform":
            arrival_times = generate_uniform_arrivals(N, lam)

        # Generate service times
        service_times = generate_service_time(N, mu)

        # Run the simulation
        service_start_times, customer_departure_time, customer_waiting_time, system_times = simulate_single_server_fifo(arrival_times, service_times)
```

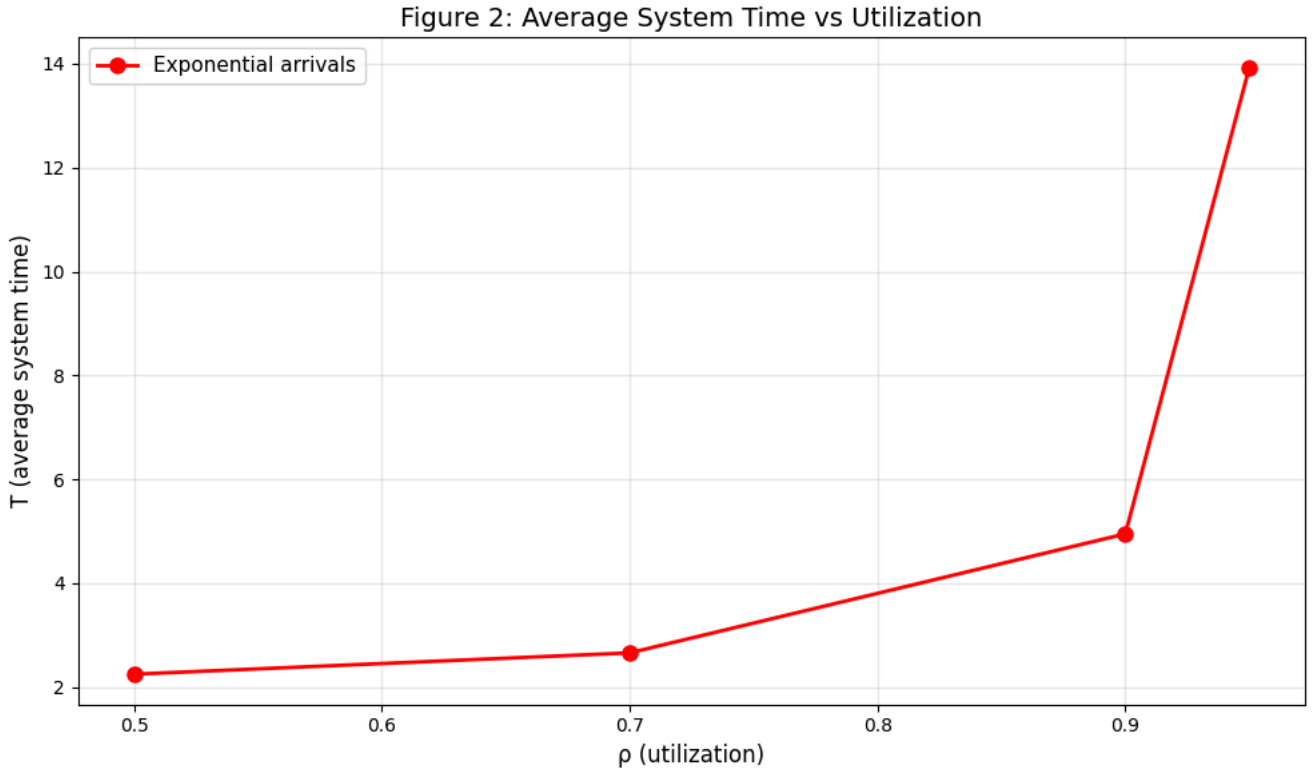
```
        # calculate metrics
        theor_util = theoritical_utilization(lam, mu)
        actual_util = actual_utilization(arrival_times, service_times, customer_departure_time)
        average_wait_time = np.mean(customer_waiting_time)
        average_system_time = np.mean(system_times)

        # Little's Law approximation
        little_law = Lq_littles_law(lam, average_wait_time)

        # Average queue length
        average_queue_length = Lq_discrete_events(arrival_times, service_start_times, customer_departure_time)

        util_theor_list.append(theor_util)
        util_pract_list.append(actual_util)
        wait_time_list.append(average_wait_time)
        sys_time_list.append(average_system_time)
        Little_law_list.append(little_law)
        average_queue_length_list.append(average_queue_length)

        # Store row data
        table_data.append({
            'lambda_rate': lam,
            'mu_rate': mu,
            'ρ_theor': theor_util,
            'ρ_actual': actual_util,
            'Average wait time': average_wait_time,
            'Average system time': average_system_time,
            'Little\'s Law': little_law,
            'Average queue length': average_queue_length
        })

    return util_theor_list, util_pract_list, wait_time_list, sys_time_list, Little_law_list, average_queue_length_list, table_data
```

```
# Run exponential case
utilization, act_utilization, average_wait_time, average_system_time, Little_law, average_queue_length, table_data = run_grid(arrival_type="exp")
```

```
def create_figure_1(utilization, average_wait_time):
    """Create Figure 1: Average Waiting Time vs Utilization"""
    plt.figure(figsize=(10, 6))
    plt.plot(utilization, average_wait_time, 'bo-', label='Exponential arrivals', linewidth=2, markersize=8)
    plt.xlabel('ρ (utilization)', fontsize=12)
    plt.ylabel('W̄ (average waiting time)', fontsize=12)
    plt.title('Figure 1: Average Waiting Time vs Utilization', fontsize=14)
    plt.legend(fontsize=11)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

create_figure_1(utilization, average_wait_time)
```




```
def create_figure_2(utilization, average_system_time):
    """Create Figure 2: Average System Time vs Utilization"""
    plt.figure(figsize=(10, 6))
    plt.plot(utilization, average_system_time, 'ro-', label='Exponential arrivals', linewidth=2, markersize=8)
    plt.xlabel('ρ (utilization)', fontsize=12)
    plt.ylabel('T̄ (average system time)', fontsize=12)
    plt.title('Figure 2: Average System Time vs Utilization', fontsize=14)
    plt.legend(fontsize=11)
    plt.grid(True, alpha=0.3)
    plt.tight_layout()
    plt.show()

create_figure_2(utilization, average_system_time)
```




```
# # Create DataFrame
# table = pd.DataFrame(table_data)

# # Create and display the table
# print("Table 1: Exponential Interarrivals Summary")
# print("=" * 60)
# table.round(4)
```

```
# If you already have table_data from run_grid()
np_table = np.array([[row[key] for key in row.keys()] for row in table_data])
# Column headers
headers = ['lambda_rate', 'mu_rate', 'ρ_theor', 'ρ_actual',
           'Average wait time', 'Average system time',
           "Little's Law", 'Average queue length']

# Print formatted table
print("Table 1: Exponential Interarrivals Summary")
print("=" * 150)
for i, header in enumerate(headers):
    print(f"{header:>18}", end="")
print()
print("-" * 150)

for row in np_table:
```

```
    for val in row:
        print(f"{val:>18.4f}", end="")
    print()
```

```
Table 1: Exponential Interarrivals Summary
=================================================================================================================================================================
       lambda_rate           mu_rate           ρ_theor          ρ_actual Average wait timeAverage system time      Little's LawAverage queue length
-----------------------------------------------------------------------------------------------------------------------------------------------------------------
            0.5000            1.0000            0.5000            0.5057            1.2433            2.2472            0.6216            0.9694
            0.7000            1.0000            0.7000            0.6635            1.6855            2.6576            1.1799            1.6880
            0.9000            1.0000            0.9000            0.8323            3.9683            4.9489            3.5715            4.1490
            0.9500            1.0000            0.9500            0.9327           12.9213           13.9211           12.2752           12.9451
```

Effect of wrong modeling.

- Keep μ fixed; keep the mean interarrival time equal to 1/λ but replace exponential interarrivals by uniform interarrivals on [0, 2/λ] (same mean, different variability).
- Re-run the experiment grid and compare metrics to the Poisson/exponential baseline

I have initially called the exponential process, below I will call the Uniform process and compare

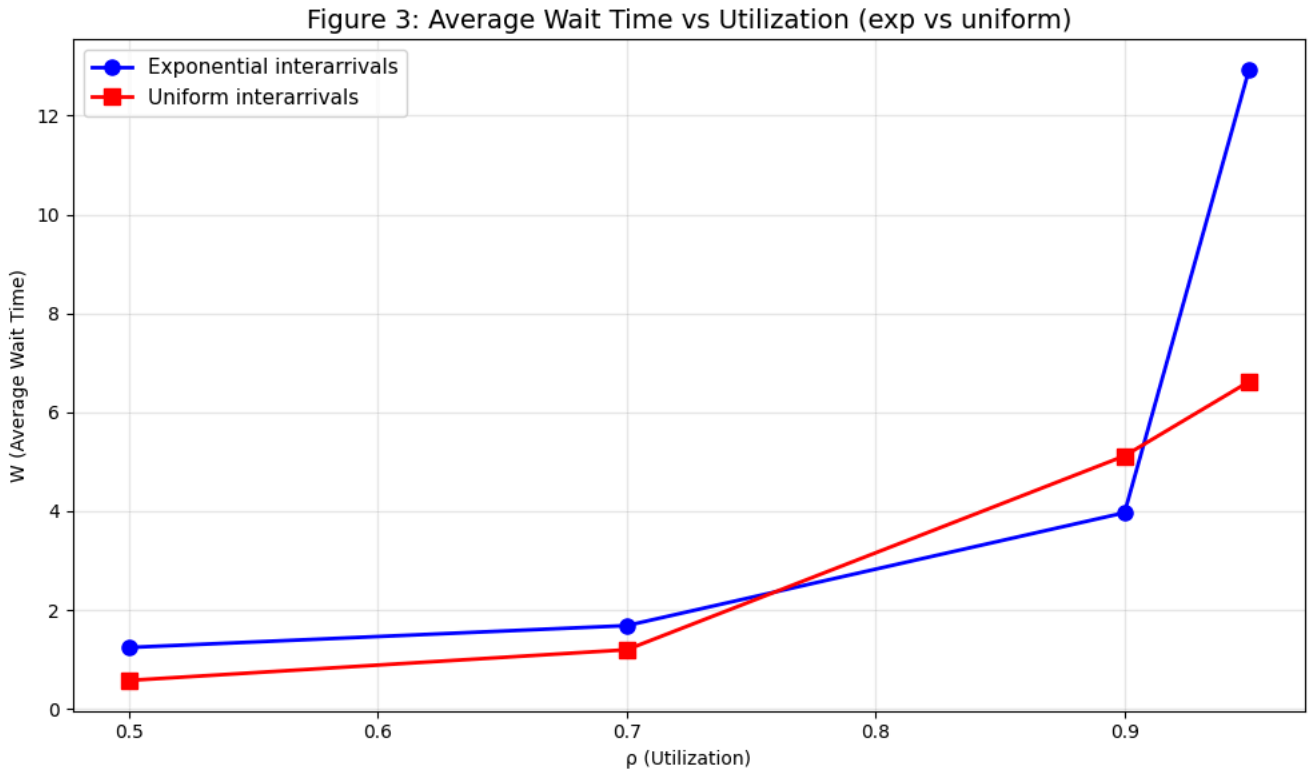
```
# Run Uniform case
uni_utilization, uni_actual_util_list, uni_average_wait_time, uni_average_system_time, uni_Little_law, uni_average_queue_length, uni_table_data = run_grid(arrival_type="uniform'
```

```
# If you already have table_data from run_grid()
uni_np_table = np.array([[row[key] for key in row.keys()] for row in uni_table_data])
# Column headers
headers = ['lambda_rate', 'mu_rate', 'ρ_theor', 'ρ_actual',
           'Average wait time', 'Average system time',
           "Little's Law", 'Average queue length']

# Print formatted table
print("Table 2: Uniform Interarrivals Summary")
print("=" * 160)
for i, header in enumerate(headers):
    print(f"{header:>18}", end="")
print()
print("-" * 160)

for row in uni_np_table:
    for val in row:
        print(f"{val:>18.4f}", end="")
    print()
```

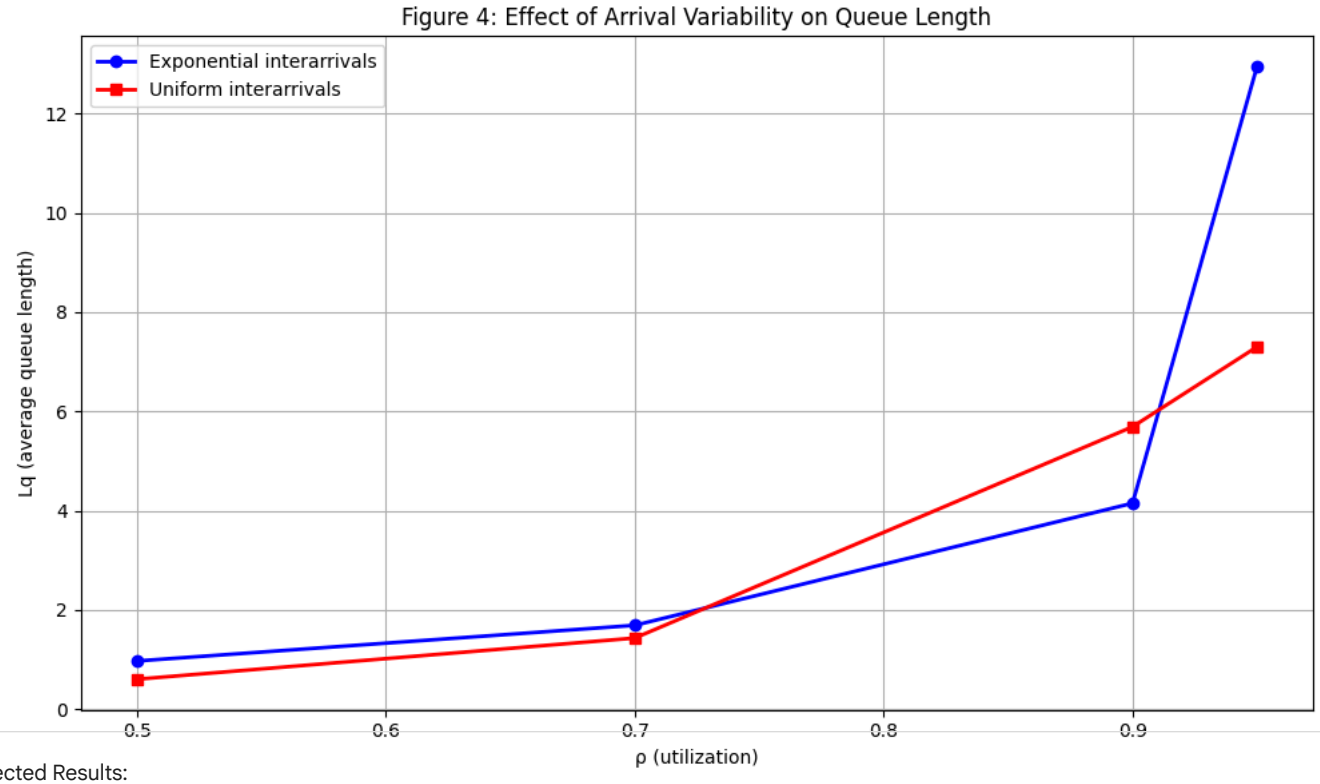
```
Table 2: Uniform Interarrivals Summary
================================================================================================================================================================
       lambda_rate           mu_rate           ρ_theor          ρ_actual Average wait timeAverage system time      Little's LawAverage queue length
----------------------------------------------------------------------------------------------------------------------------------------------------------------
            0.5000            1.0000            0.5000            0.5101            0.5772            1.6162            0.2886            0.6018
            0.7000            1.0000            0.7000            0.7021            1.1974            2.1985            0.8382            1.4316
            0.9000            1.0000            0.9000            0.9148            5.1177            6.1107            4.6059            5.6896
            0.9500            1.0000            0.9500            0.9407            6.6187            7.6104            6.2877            7.3004
```

```
Start coding or generate with AI.
```

```
# Figure 3: average wait time vs utilization (exp vs uniform)
plt.figure(figsize=(10, 6))
plt.plot(utilization, average_wait_time, 'bo-', label='Exponential interarrivals', linewidth=2, markersize=8)
plt.plot(uni_utilization, uni_average_wait_time, 'rs-', label='Uniform interarrivals', linewidth=2, markersize=8)
plt.xlabel('ρ (Utilization)')
plt.ylabel('W (Average Wait Time)')
plt.title('Figure 3: Average Wait Time vs Utilization (exp vs uniform)', fontsize=14)
plt.legend(fontsize=11)
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```




```
# Figure 4: Lqbar vs rho (exp vs uniform)
plt.figure(figsize=(10, 6))
plt.plot(utilization, average_queue_length, 'bo-', label='Exponential interarrivals', linewidth=2)
plt.plot(uni_utilization, uni_average_queue_length, 'rs-', label='Uniform interarrivals', linewidth=2)
plt.xlabel('ρ (utilization)')
plt.ylabel('Lq (average queue length)')
plt.title('Figure 4: Effect of Arrival Variability on Queue Length')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



Expected Results:

- Exponential arrivals should show higher waiting times and queue lengths
- Uniform arrivals should show lower waiting times and queue lengths
- Difference increases dramatically as ρ approaches 1

Short Discussion: Impact of Arrival Variability on System Design

The comparison between exponential and uniform interarrivals with identical means reveals that arrival variability has a significant impact on queueing system performance, particularly at high utilization levels. Exponential interarrivals create random clustering where customers arrive in unpredictable bursts followed by gaps, leading to congestion and longer queues, while uniform interarrivals provide perfectly predictable spacing that eliminates clustering effects and maintains smoother system operation. As shown in Figure 4, this difference becomes increasingly critical as utilization approaches 1, where exponential arrivals produce nearly double the queue length ($L_q \approx 14$) compared to uniform arrivals ($L_q \approx 7.5$) at $\rho = 0.95$.

The "wrong modeling" effect demonstrates a dangerous system design pitfall: if engineers assume low-variability uniform arrivals when the true process follows high-variability exponential arrivals, they will severely underestimate the required queue capacity and service levels needed for acceptable performance. At high utilization levels, this modeling error becomes catastrophic, as a system designed for uniform arrivals would require almost double the queue space and buffer capacity to handle the reality of exponential arrival clustering. Such wrong assumptions could lead to system overloads, unacceptable customer wait times, and service failures in real-world applications where arrival variability is an inherent characteristic of the demand process.

```
Start coding or generate with AI.
```